WEB SECURITY ACADEMY · PORTSWIGGER

Username Enumeration via Different Responses

Resolución guiada paso a paso de un laboratorio de autenticación: enumeración de usuarios y fuerza bruta de contraseña con Burp Suite

Creado por

vareCruzz

Nivel: Aprendiz · Categoría: Authentication · Herramienta: Burp Suite

Introducción al laboratorio

Este documento explica, de forma clara y detallada, la resolución del laboratorio de PortSwigger «Username enumeration via different responses» (Enumeración de nombres de usuario a través de diferentes respuestas). Es un laboratorio de nivel Aprendiz dentro de la categoría de vulnerabilidades de autenticación.

El objetivo declarado es el siguiente: el formulario de login es vulnerable a enumeración de nombres de usuario y a ataques de fuerza bruta de contraseña. Existe una cuenta cuyo usuario y contraseña son predecibles y se encuentran dentro de dos listas de palabras candidatas. Para resolver el laboratorio hay que:

- Enumerar un nombre de usuario válido dentro de la aplicación.
- Realizar fuerza bruta sobre la contraseña de ese usuario.
- Iniciar sesión y acceder a «My account» de dicho usuario.

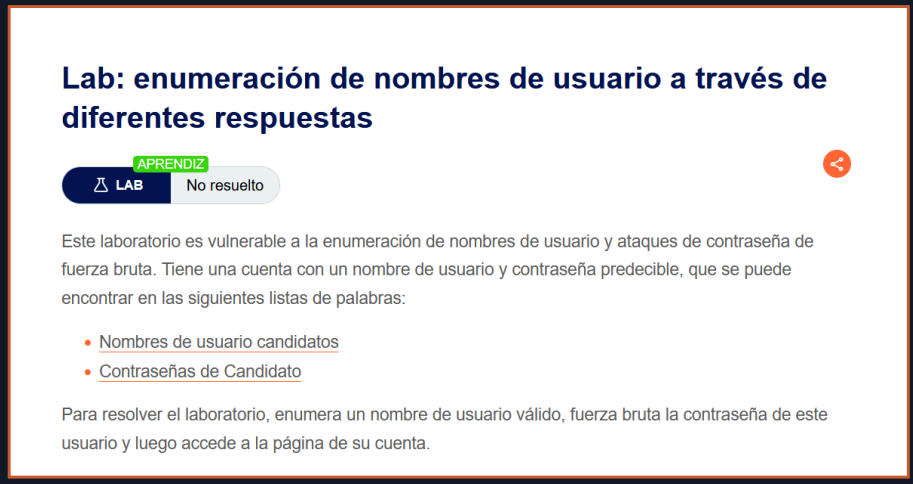


Figura 1. Descripción oficial del laboratorio en PortSwigger Web Security Academy.

Idea central de la vulnerabilidad: el servidor no responde igual cuando el usuario no existe que cuando existe pero la contraseña es incorrecta. Esa diferencia de comportamiento (mensaje, código de estado, longitud o tiempo de respuesta) es un canal lateral que permite distinguir usuarios reales sin conocer ninguna contraseña.

02 Resolución paso a paso

A continuación se detalla el proceso completo, apoyado en las capturas reales de la sesión, usando el proxy de interceptación Burp Suite y su módulo Intruder.

1 Probar una petición de login e interceptarla

Lo primero es abrir la página de login e introducir unas credenciales cualquiera a modo de prueba (usuario `root`). El objetivo no es acertar, sino generar una petición `POST /login` real que capturar con el proxy de Burp Suite y reenviar al módulo Intruder.

> Pista clave: la aplicación responde explícitamente con «Invalid username». Ese mensaje se usará para distinguir usuarios válidos de inválidos de forma automatizada.

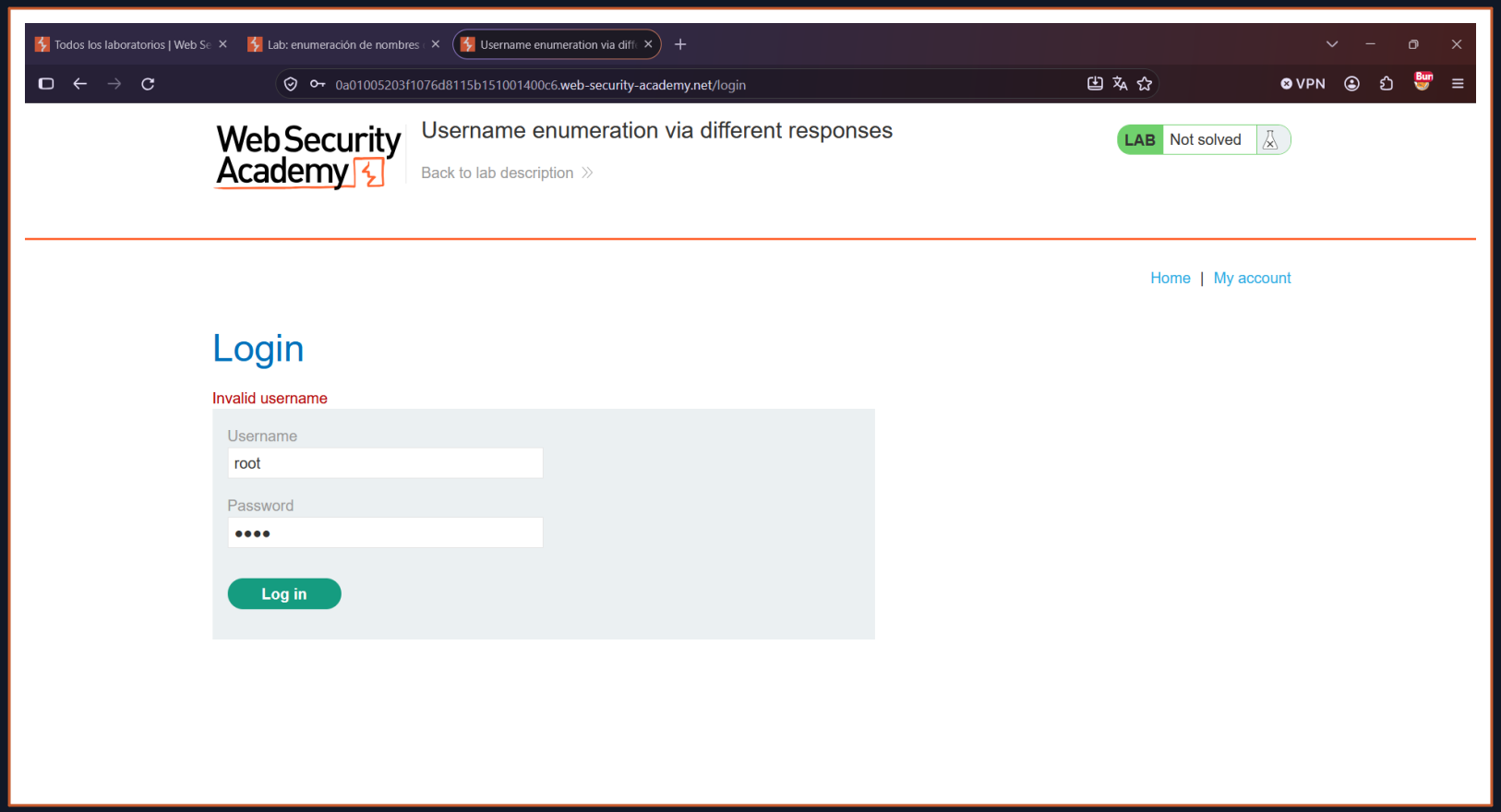


Figura 2. El servidor responde «Invalid username» ante un usuario inexistente.

2 Enviar la petición a Intruder y cargar la lista de usuarios

Con Burp interceptando el tráfico, se envía la petición al módulo Intruder. Se selecciona el ataque Sniper, se marca como posición de payload el parámetro `username` (contraseña fija), y en «Payloads» se carga la lista de candidate usernames del laboratorio (101 nombres candidatos).

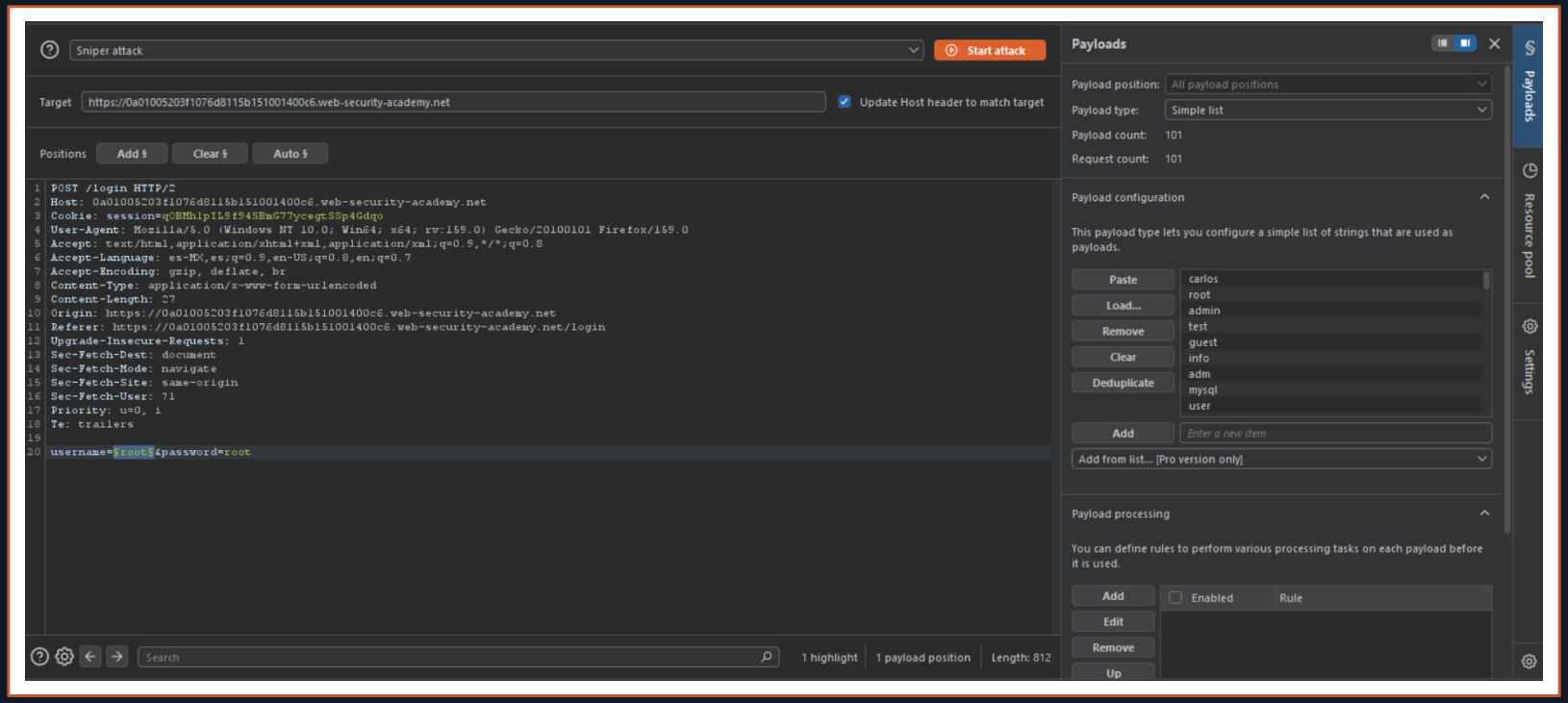


Figura 3. Posición de payload en «username» con 101 candidatos cargados.

3 Comparar las respuestas: usuario válido vs. inválido

Al ejecutar el ataque, Burp envía una petición por cada usuario candidato. La clave está en comparar el contenido de cada respuesta, no solo el código HTTP (200 en casi todos los casos). Cuando el usuario existe, el servidor responde «Incorrect password»; cuando no existe, responde «Invalid username».

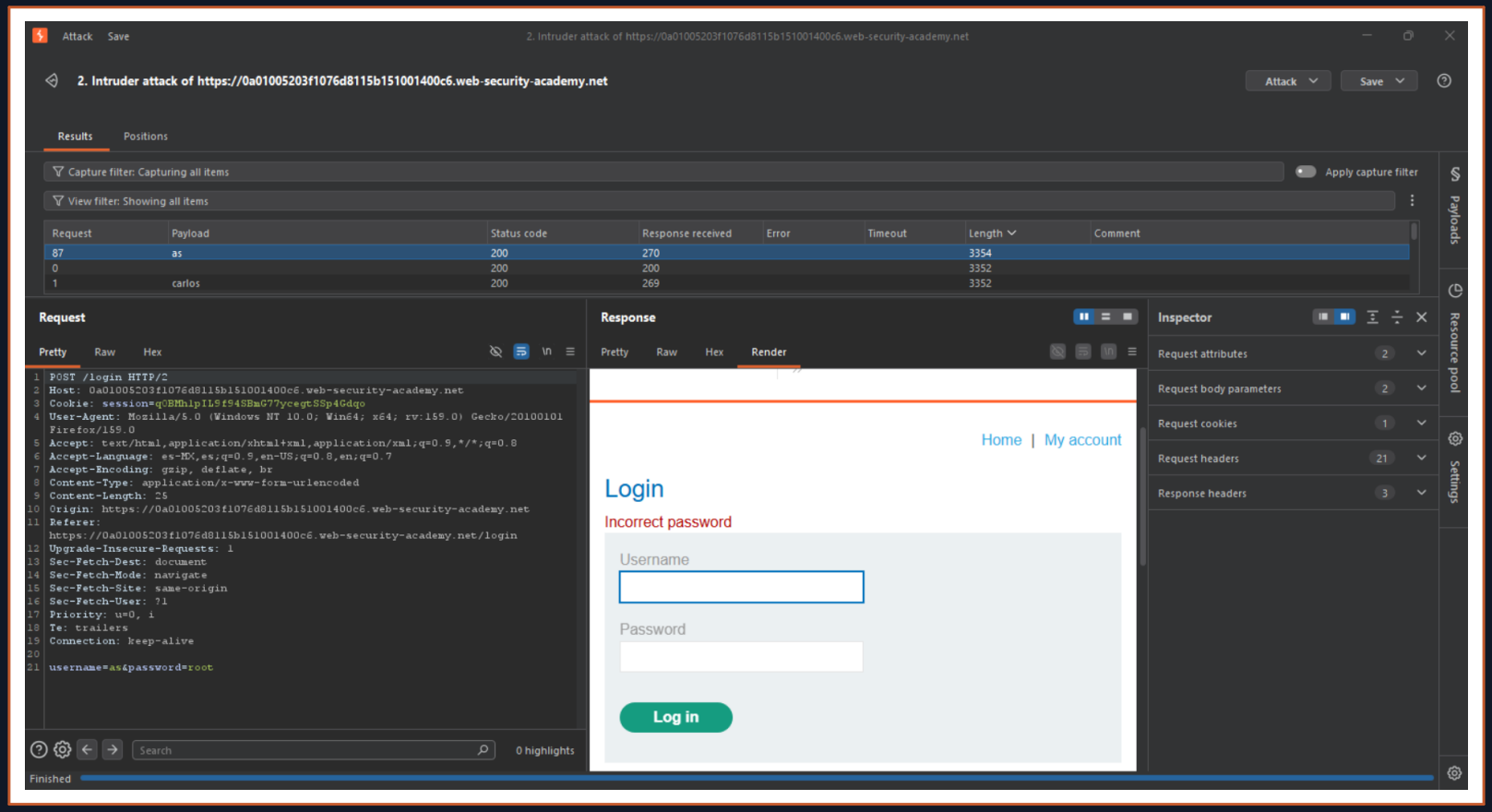


Figura 4. Usuario «carlos» → «Incorrect password»: el usuario existe.

4 Confirmar el patrón con un usuario inválido

Para contrastar, se revisa la respuesta de un usuario que sí es inválido, como `root`: el mensaje cambia a «Invalid username». Esta diferencia textual entre ambos mensajes de error es exactamente el fallo que da nombre a la vulnerabilidad de enumeración de usuarios.

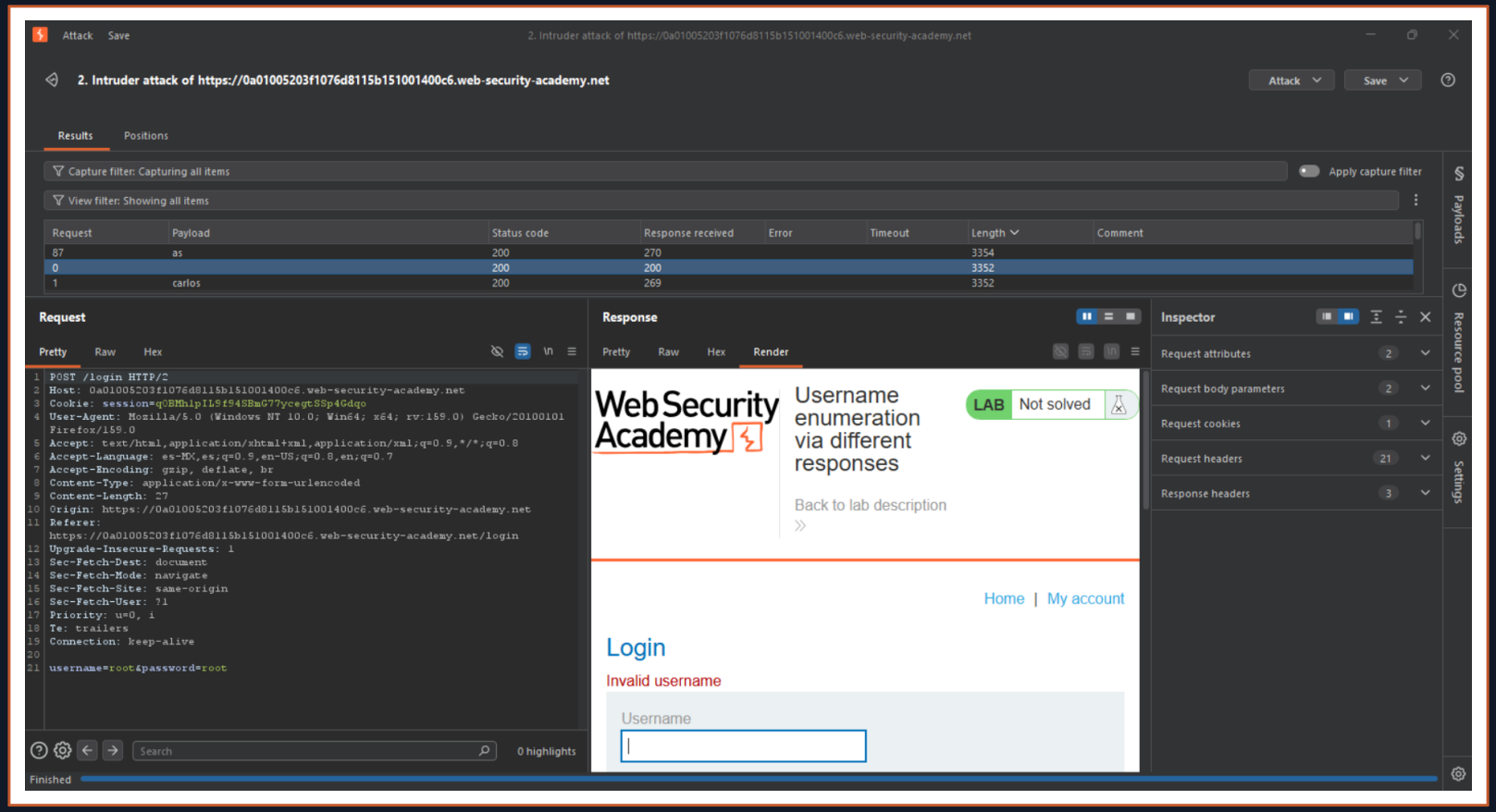


Figura 5. Usuario «root» → «Invalid username»: el usuario no existe.

5 Ordenar por longitud de respuesta para hallar al usuario real

En vez de revisar 101 respuestas a mano, se ordena la tabla de Intruder por la columna Length. Como los dos mensajes de error tienen longitudes de texto distintas, esto agrupa las respuestas en dos bloques y delata de inmediato qué payload rompe el patrón general.

Resultado: el nombre de usuario válido es as.

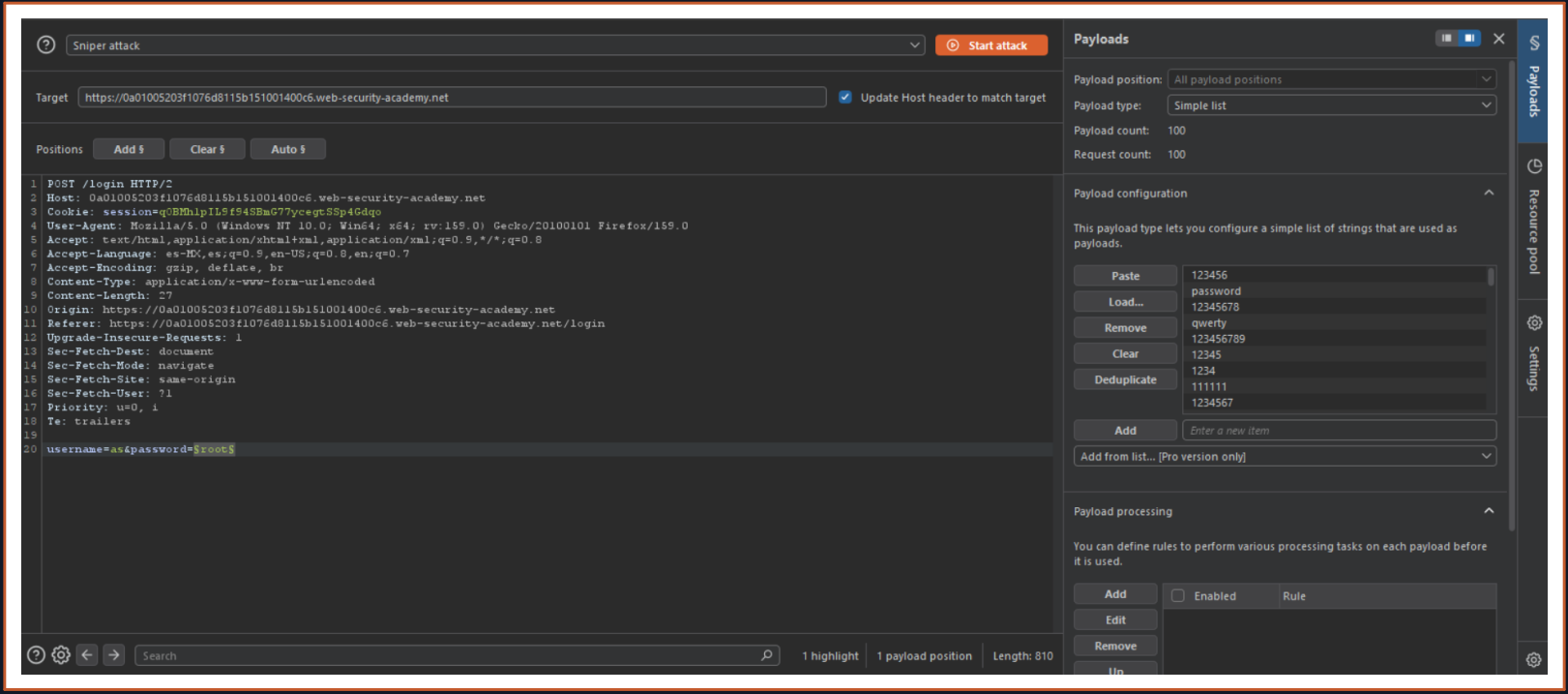


Figura 6. El payload «as» produce 3354 bytes frente a 3352 del resto: usuario real confirmado.

6 Repetir Intruder sobre el campo «password»

Con el usuario confirmado, se repite el ataque fijando `username=as` y marcando como posición de payload el parámetro `password`, usando la segunda lista del laboratorio: 100 candidate passwords comunes.

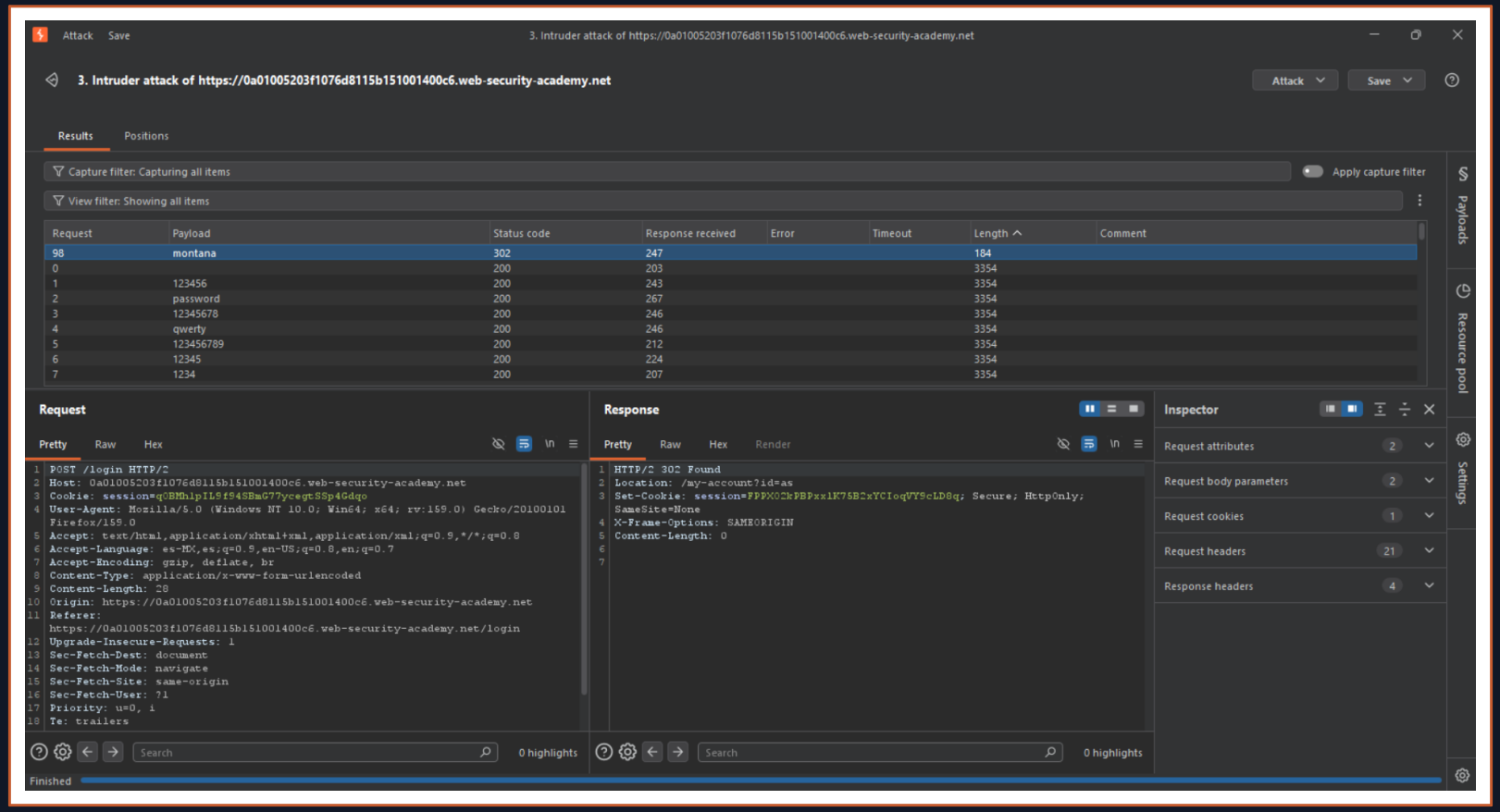


Figura 7. Usuario fijo «as», payload sobre «password» con 100 contraseñas candidatas.

7 Detectar la respuesta 302 (login correcto)

Todas las contraseñas incorrectas devuelven `HTTP 200` con «Incorrect password». La contraseña correcta, en cambio, provoca un `HTTP 302 Found` con cabecera `Location: /my-account?id=as`, el comportamiento típico tras un login exitoso.

Resultado: la contraseña correcta es `montana`.

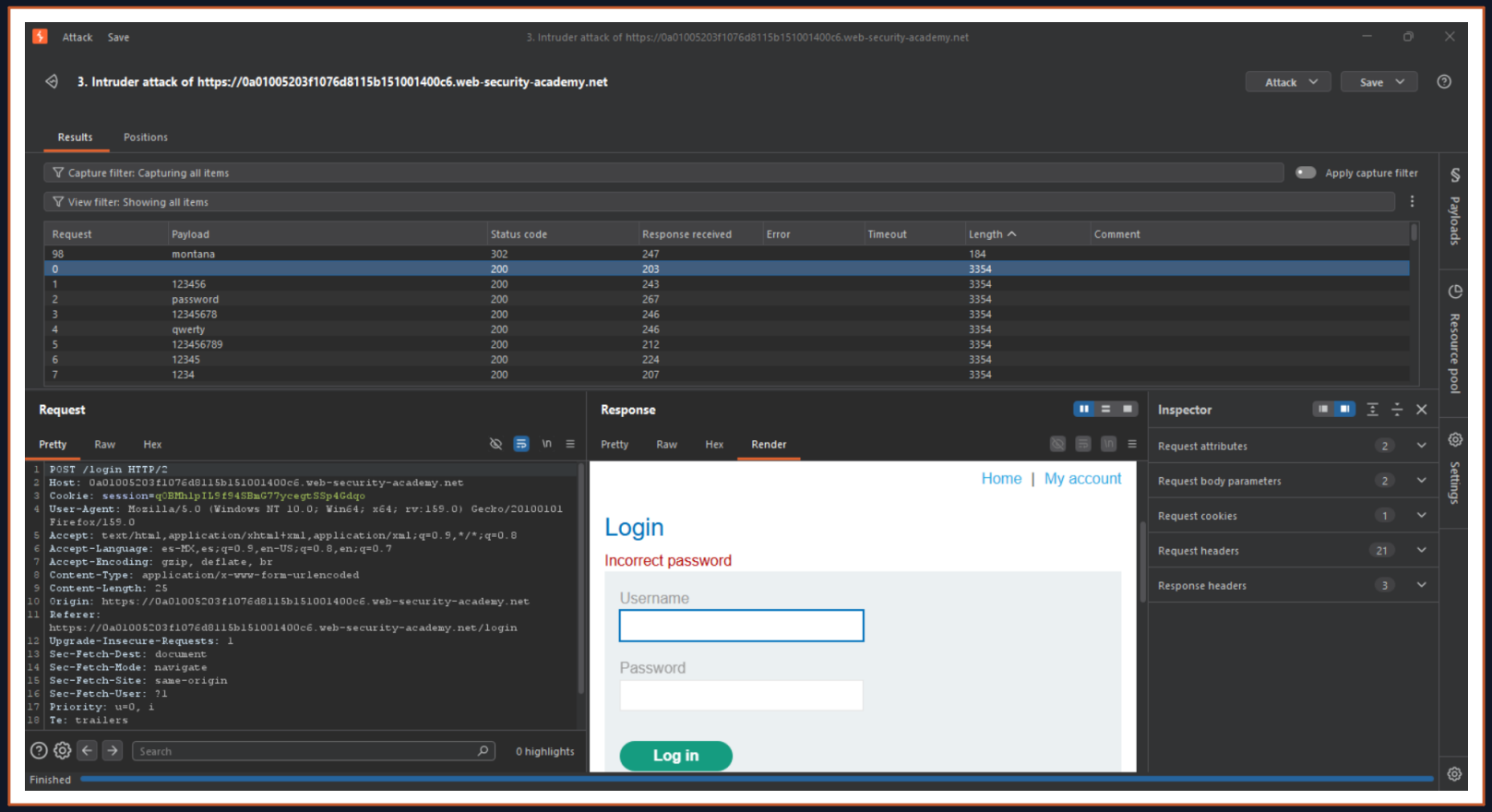


Figura 8. El payload «montana» devuelve 302 Found: contraseña correcta hallada.

8 Iniciar sesión manualmente y confirmar la resolución

Con las credenciales `as` / `montana` descubiertas, se vuelve al formulario de login y se introducen manualmente. La aplicación muestra «My Account» ya autenticado y el laboratorio cambia su estado a «Solved».

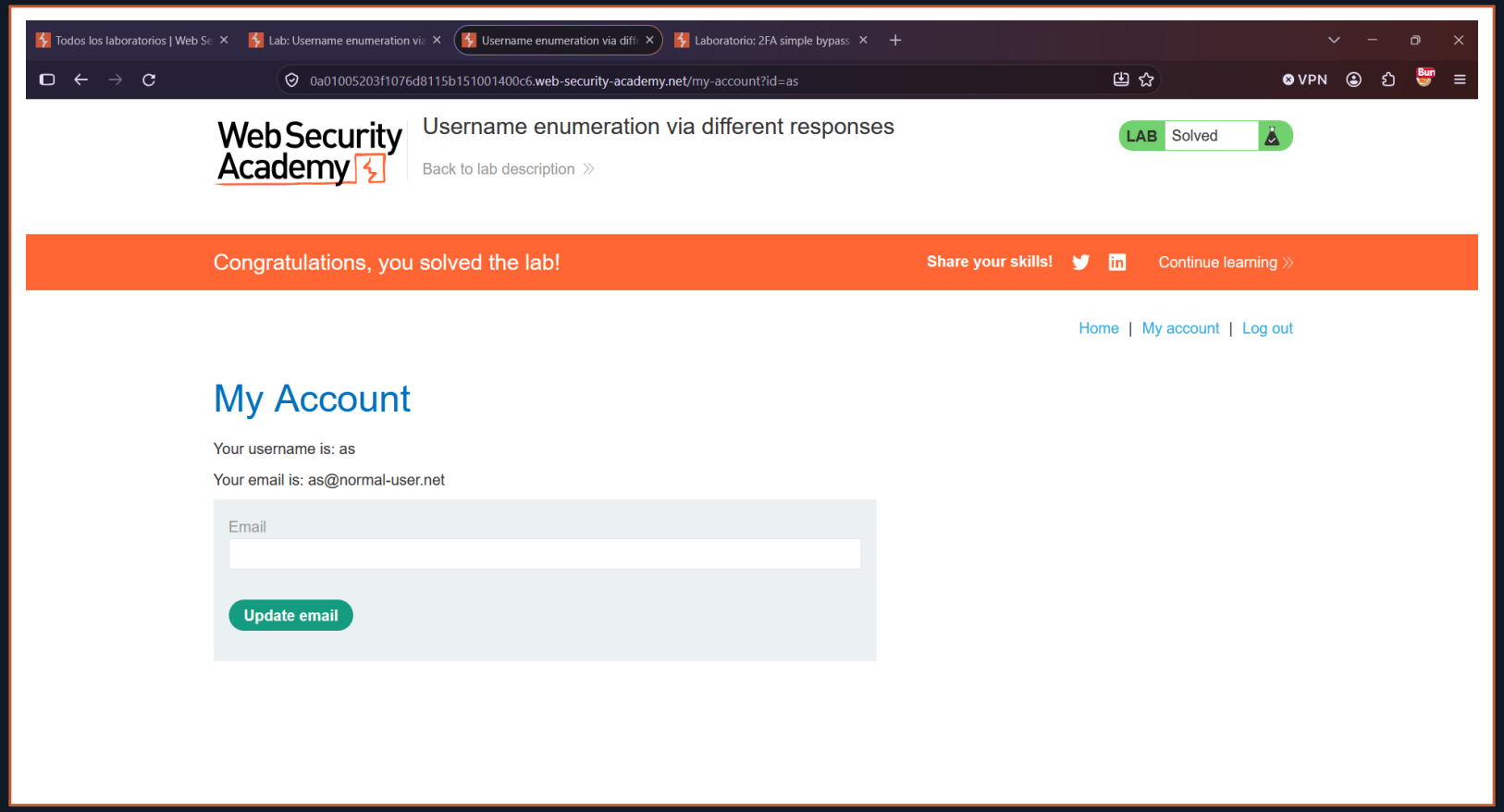


Figura 9. Acceso exitoso — el laboratorio se marca como «Solved».

Resumen del flujo de ataque

Paso	Acción	Resultado
1	Fuerza bruta de username con contraseña fija	Usuario válido: as
2	Fuerza bruta de password con usuario fijo «as»	Contraseña válida: montana
3	Login manual con «as» / «montana»	Acceso concedido — Lab resuelto

03 ¿Por qué ocurrió esta vulnerabilidad?

La causa raíz es un fallo de diseño muy común en formularios de autenticación: el servidor revela, de forma involuntaria, si un nombre de usuario existe o no a través de diferencias observables en sus respuestas HTTP — lo que se conoce como «user enumeration». Puede manifestarse de varias formas:

- Mensajes de error distintos, como en este laboratorio.
- Códigos de estado HTTP distintos (200 vs. 404/401).
- Diferencias de tiempo de respuesta (hash vs. búsqueda fallida).
- Diferencias mínimas de longitud, aunque el texto parezca idéntico.

El desarrollador implementó la validación en dos pasos secuenciales — «¿existe el usuario?» y, solo si la respuesta es sí, «¿es correcta la contraseña?» — dejando que cada paso devolviera una respuesta reconocible. Esto convierte el login en un oráculo: un atacante puede reducir la lista de cuentas reales en segundos, sin conocer ninguna contraseña.

Una vez confirmado un usuario real, la fuerza bruta de contraseña se vuelve mucho más eficiente: ya no se prueban combinaciones usuario × contraseña, sino solo contraseñas contra un usuario conocido. Sin límite de intentos, bloqueo de cuenta o CAPTCHA —como en este laboratorio—, un atacante puede probar miles de contraseñas por segundo.

Recomendaciones de mitigación

- Mensaje genérico idéntico para usuario o contraseña incorrectos.
- Tiempo de respuesta constante exista o no el usuario.
- Longitud y estructura de respuesta idénticas en ambos casos.
- Rate limiting, bloqueo temporal o CAPTCHA tras varios fallos.
- Monitorizar patrones de tráfico anómalos de login.

Documento elaborado por vareCruzz como ejercicio de aprendizaje en seguridad web ofensiva, basado en el laboratorio «Username enumeration via different responses» de PortSwigger Web Security Academy.